



uGDX School of Technology

Mini Project Report on

“StudySync – Cloud-Based Assignment Manager for Students”

Submitted in partial fulfillment of the
requirements for the course

**Course: Cloud Application Development (CAP
303) Class: TY B. Tech (Semester V)**

By

Kuunal Mistry 2410130

Supervisor:

Dr. Kunal Meher
(Assistant Professor)

AY: 2026-2027



uGDX School of Technology

CERTIFICATE

This is to certify that the project entitled “**StudySync – Cloud-Based Assignment Manager for Students**” is a bonafide work of

Kuunal Mistry

submitted in fulfilment of the requirement for the course “**Cloud Application Development**” of Third Year of Bachelor degree in Technology (CS, AI & ML) during academic year 2026-27.

Supervisor/Guide External Examiner (Dr. Kunal Meher)

Table of Contents

1. Introduction.....	1
2. Technologies Used.....	2
a. Development Technology Used.....	2 b.
Deployment Technology Used	2 3.
Code.....	6 4.
Screenshots	9 5.
Conclusion	14

1. Introduction

StudySync is a cloud-based assignment management web application developed for students. The application provides a centralized interface for organizing subjects and assignments while also providing file-management functionality for academic documents.

The project was developed using **Python and Flask** for the application layer. **Azure** is used for application hosting, database services, and monitoring. **Amazon Web Services (AWS) S3** is used for object/file storage, while **Cloudflare Worker** provides the edge and reverse-proxy layer.

The project follows a **multi-cloud architecture** in which different services are used according to their role:

- Cloudflare handles the edge request path.
- Azure hosts the application and structured data.
- AWS S3 stores uploaded files.
- Azure Application Insights provides observability and monitoring.

Final Request Flow

User → Cloudflare Worker → Azure App Service → Azure MySQL / AWS S3
Application Insights → monitors the application

Project Objectives

- Provide a centralized web application for managing student assignments and subjects. • Support creation, editing, deletion, and organization of assignments. • Allow assignment-related files to be uploaded, downloaded, and deleted. • Use cloud services for application hosting, database management, file storage, edge routing, and monitoring.
- Demonstrate a practical multi-cloud deployment using Azure, AWS, and Cloudflare.

Major Features

- **Assignment management:** Create, view, edit, and delete assignments. • **Subject organization:** Organize assignments according to subjects. • **File management:** Upload, download, and delete assignment files.
- **Cloud hosting:** Run the Flask application on Azure App Service.
- **Database:** Store structured application data in Azure Database for MySQL. • **Monitoring:** Monitor application performance, requests, errors, and logs using Azure Application Insights.
- **Edge routing:** Use Cloudflare Worker as the public edge/reverse-proxy endpoint.

2. Technologies Used

a. Development Technology Used

Technology	Purpose
Python	Primary programming language used for the application.
Flask	Web framework used to build the StudySync application and routes.
Flask-SQLAlchemy	ORM layer used for database models and database operations.
PyMySQL	MySQL database driver used by the application.
Boto3	AWS SDK used by the Flask application to upload, download, and delete objects in Amazon S3.
HTML / CSS	Used to build the application pages and user interface.
python-dotenv	Used to load environment variables for local configuration.
Git / GitHub	Used for source-code version control and repository management.

b. Deployment Technology Used

Cloud Service	Role in StudySync
Azure App Service	Hosts and runs the Flask StudySync application.
Azure Database for MySQL Flexible Server	Stores structured application data such as assignments, subjects, and application records.
Azure Application Insights	Collects application performance telemetry, requests, errors, and logs.
Amazon S3	Provides private object storage for uploaded assignment files.
Cloudflare Worker	Acts as the public edge/reverse-proxy layer and forwards requests to Azure App Service.

Cloud Architecture

The production architecture uses five cloud services.

Azure App Service hosts and runs the Flask StudySync application. It runs the web application, handles user requests, connects to Azure MySQL, and accesses Amazon S3 for file operations.

Azure Database for MySQL provides the relational database for StudySync. It stores structured application information such as:

- Assignments
- Subjects
- Application records
- User-related application data
- Settings and other database-backed information

Amazon S3 provides object storage for uploaded files. It is used to store uploaded assignment files, retrieve files when users request downloads, and delete files when they are removed from the application. The bucket is private and accessed by the Flask application through the AWS SDK.

Cloudflare Worker acts as the public edge/reverse-proxy layer. It receives requests from the public StudySync endpoint, forwards them to Azure App Service, preserves the request path and query parameters, and keeps the Azure application origin behind the Worker endpoint.

Azure Application Insights provides monitoring and diagnostics. It is used to monitor application performance, track requests, track errors and failed requests, collect application logs and telemetry, and help with debugging and analysis.

Architecture Flow

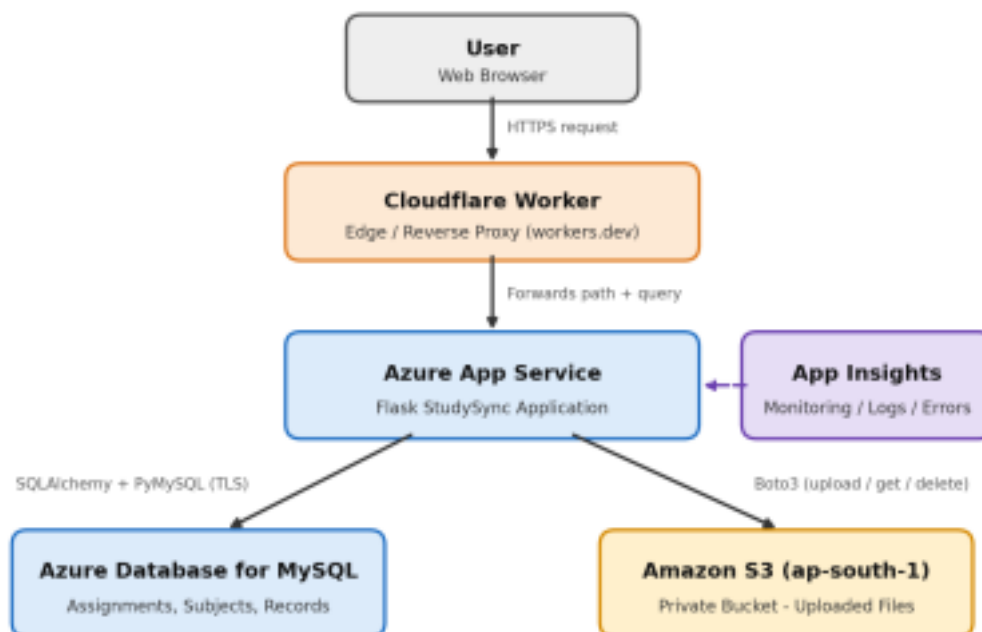


Figure 2.1: StudySync multi-cloud architecture (Cloudflare + Azure + AWS)

Overall request flow:



Resource Configuration Summary

Resource	Name / Value
Azure Resource Group	StudySync-RG
Azure App Service	studysync-kuunalmistry
Azure MySQL Flexible Server	studysync-mysql-kuunal
Azure Application Insights	StudySync-Insights
AWS S3 Bucket	studysync-kuunalmistry-files
AWS Region	ap-south-1 — Mumbai
Cloudflare Worker	studysync
Public StudySync Endpoint	https://studysync.kuunalmistry.workers.dev

Deployment Verification Log

The Azure deployment was completed using Azure CLI ZIP deployment. The successful deployment returned:

```
Status: Site started successfully.
```

```
"status": "RuntimeSuccessful"
```

```
"numberOfInstancesSuccessful": 1
```

```
"errors": null
```

This verified that the application was successfully deployed and the App Service runtime started correctly.

Testing Performed

The following tests were performed during development and deployment.

Azure MySQL

- Local database connectivity was tested against Azure MySQL.

- Database tables were created successfully.
- A test assignment was created successfully.
- The test assignment was deleted successfully.

Amazon S3

- File upload was tested.
- Uploaded files were verified in the S3 bucket.
- File download was tested.
- File deletion was tested.

Azure App Service

- StudySync was deployed successfully.
- The App Service runtime started successfully.
- The application was accessed through the production deployment.

Cloudflare Worker

- The Worker successfully proxied requests to Azure App Service.
- The StudySync dashboard loaded through the Worker URL.
- Static CSS assets loaded correctly.
- Production application functionality was tested through the Worker endpoint.

Application Insights

Application telemetry was verified, including server requests, response time, failed requests, and application monitoring data.

A 404 request for `robots933456.txt` was observed in telemetry. This was an external request for a nonexistent file and did not represent a StudySync application failure.

3. Code

The project source code is organized as a Flask application with HTML templates, CSS assets, database models, and cloud-service integrations. Sensitive credentials are stored in environment variables and are **not included in the GitHub repository**.

Project Structure

```
StudySync/
├── certs/
├── static/
│   └── style.css
├── templates/
│   ├── _sidebar.html
│   ├── assignments.html
│   ├── edit_assignment.html
│   ├── files.html
│   ├── index.html
│   ├── settings.html
│   ├── subject_assignments.html
│   └── subjects.html
├── uploads/
├── .env
├── .gitignore
├── app.py
├── README.md
├── requirements.txt
└── schema.sql
```

Important Files

File / Folder	Description
app.py	Main Flask application containing Flask configuration, database models, application routes, assignment operations, subject operations, file operations, and AWS S3 integration.
templates/	Contains the HTML templates used by the StudySync interface.
static/	Contains CSS and front-end assets.
requirements.txt	Contains the Python dependencies required to run the application.
schema.sql	Contains the database schema/reference SQL.
.env	Contains local environment configuration and sensitive credentials. Excluded from Git.
certs/	Contains the local database TLS certificate used for secure database connectivity. Certificate files are excluded from Git.

Database Configuration

StudySync uses **SQLAlchemy with PyMySQL** to connect the Flask application to Azure Database for MySQL. The database connection values are supplied through environment variables such as `DB_HOST`, `DB_NAME`, `DB_PORT`, and `DB_PASSWORD`. A simplified representation of the database configuration is:

```
app.config["SQLALCHEMY_DATABASE_URI"] = (  
    "mysql+pymysql://<user>:<password>@<host>:3306/<database>"  
)  
  
db = SQLAlchemy(app)
```

The actual database credentials are kept private in environment variables and are not committed to GitHub.

AWS S3 File Storage

Uploaded files are stored in the private Amazon S3 bucket rather than being used as persistent local application storage. The Flask application uses **Boto3** to:

1. Upload objects to S3.
2. Retrieve objects for download.
3. Delete objects when the corresponding file record is removed.

Representative operations are:

```
s3.upload_fileobj(file, S3_BUCKET_NAME, s3_key)  
  
obj = s3.get_object(  
    Bucket=S3_BUCKET_NAME,  
    Key=stored_filename  
)  
  
s3.delete_object(  
    Bucket=S3_BUCKET_NAME,  
    Key=stored_filename  
)
```

The application records the S3 object key in the database so that the correct file can be retrieved or deleted later.

Cloudflare Worker

The Cloudflare Worker receives requests from the public StudySync `workers.dev` endpoint and forwards the request path and query string to the Azure App Service origin. Simplified Worker logic:

```
const targetUrl =
  AZURE_ORIGIN + incomingUrl.pathname + incomingUrl.search;

return fetch(new Request(targetUrl, {
  method: request.method,
  headers: request.headers,
  body: ["GET", "HEAD"].includes(request.method)
    ? undefined
    : request.body
}));
```

This creates the following flow: **Public StudySync URL → Cloudflare Worker → Azure App Service**.

Version Control and Security

The project was initialized as a Git repository at the StudySync project root and pushed to GitHub. The repository contains source code and project documentation.

GitHub repository: <https://github.com/kuunalmistry/StudySync>

Sensitive and environment-specific files are excluded through `.gitignore`,

- including:
- `.env`
 - Environment secrets
 - Database certificates
 - Uploaded files
 - Python cache files
 - Local development/tool-specific folders

AWS credentials and database passwords are therefore not committed to the repository.

4. Screenshots

The following screenshots serve as project evidence of the StudySync application and its cloud services.

The first screenshot shows the 'Overview' page of the Azure Database for MySQL flexible server 'studysync-mysql-kuunal'. The page displays various settings and a checklist for getting started.

Subscription: Azure for Students
Subscription ID: 5a1d5514-79d0-4725-8987-bf754d9c6443
Resource group: StudySync-RG
Status: Ready
Location: UAE North

Endpoint: studysync-mysql-kuunal.mysql.database.azure.com
Administrator login: studysyncadmin
Configuration: Burstable_B1ms_1_vCores_2_GiB_RAM_20_storage_360_IOPS
MySQL version: 8.4
Availability zone: 1
Created on: 2026-09-30 16:21:47.0785184 UTC

The second screenshot shows the 'Getting started' checklist for the same database instance. The checklist includes the following steps:

- Learn about Azure Database Migration Service
- Create migration service

The third screenshot shows the MySQL command prompt interface. The command 'SHOW DATABASES;' has been executed, and the output shows the following databases:

```
Database
+-----+
| information_schema |
| mysql               |
| performance_schema |
| student_manager     |
| sys                 |
+-----+
5 rows in set (0.034 sec)
```

Home > Microsoft.Web-WebApp-Portal-cacff741-aff0 | Overview

studysync-kuunalmistry

Web App

Search

Overview

- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Settings
- App Service plan
- Development Tools
- Monitoring
- Automation
- Support + troubleshooting

Essentials

Resource group (move) : [StudySync-RG](#)

Status : Running

Location (move) : UAE North

Subscription (move) : [Azure for Students](#)

Subscription ID : Sa1d5514-79d0-4725-8987-bf754d9c6443

Tags (edit) : [Add tags](#)

Default domain : [studysync-kuunalmistry-b3f5a9aceuhf4gy.uaenorth-01.azurewe...](#)

App Service Plan : [ASP-StudySyncRG-b72e \(F1: 1\)](#)

Operating System : Linux

Health Check : [Cannot fetch health check data. Please try again later.](#)

Properties Monitoring Logs Capabilities Notifications Recommendations

Web app

Name : studysync-kuunalmistry

Publishing model : Code

Runtime Stack : Python - 3.11

Deployment Center

Deployment logs : [View logs](#)

Last deployment : Loading deployments...

Deployment provider : None

Domains

Default domain : [studysync-kuunalmistry-b3f5a9aceuhf4gy.uaenorth-01.azurewebsites.net](#)

Custom domain : [Add custom domain](#)

Application Insights

Name : [Enable Application Insights](#)

Networking

Inbound IP addresses : 20.38.138.9, 2603:1040:904:3::31c

Private endpoint connections : Not connected

File Edit Selection View Go Run Terminal Help

StudySync

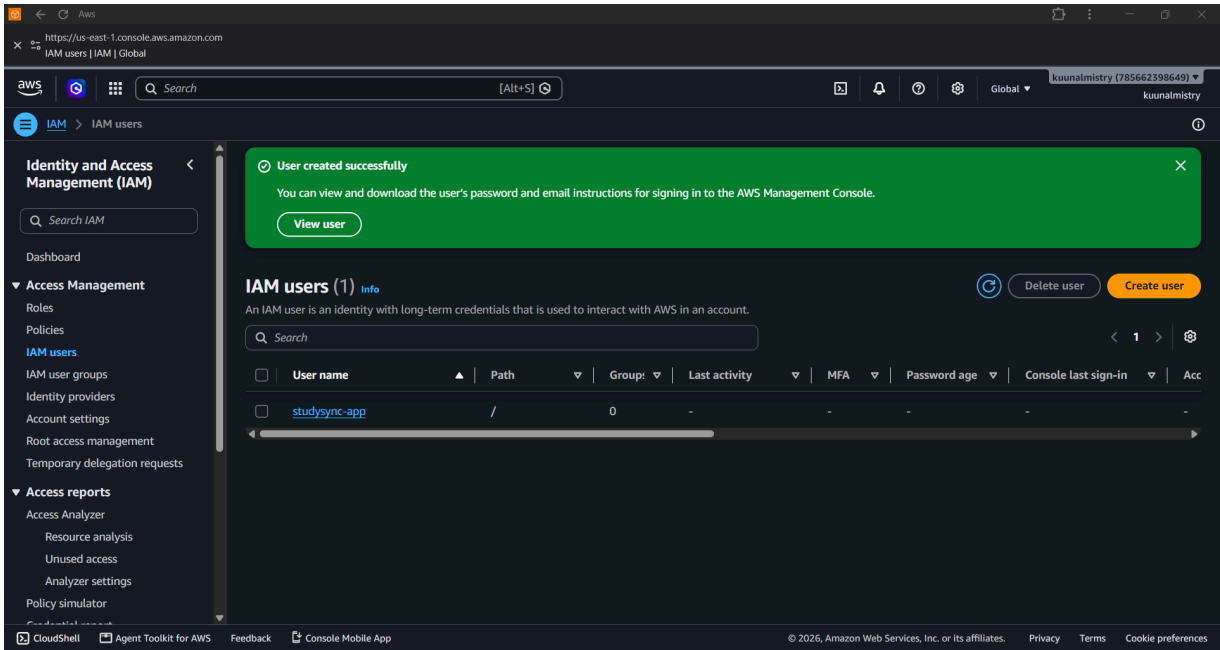
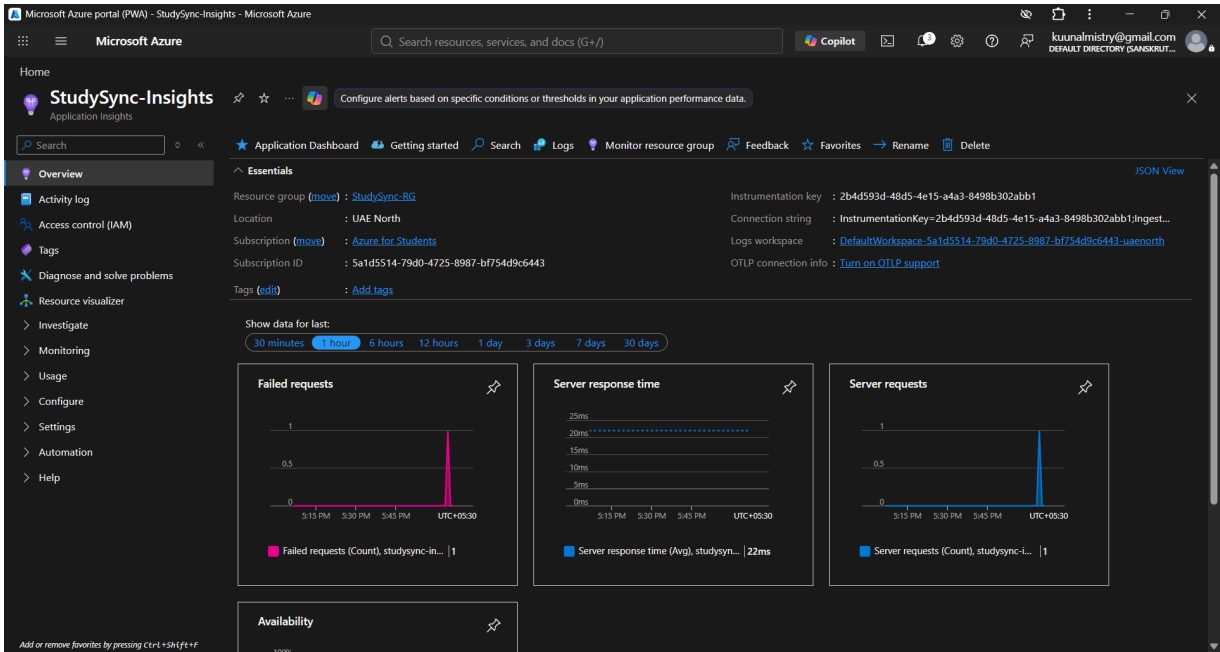
requirements.txt

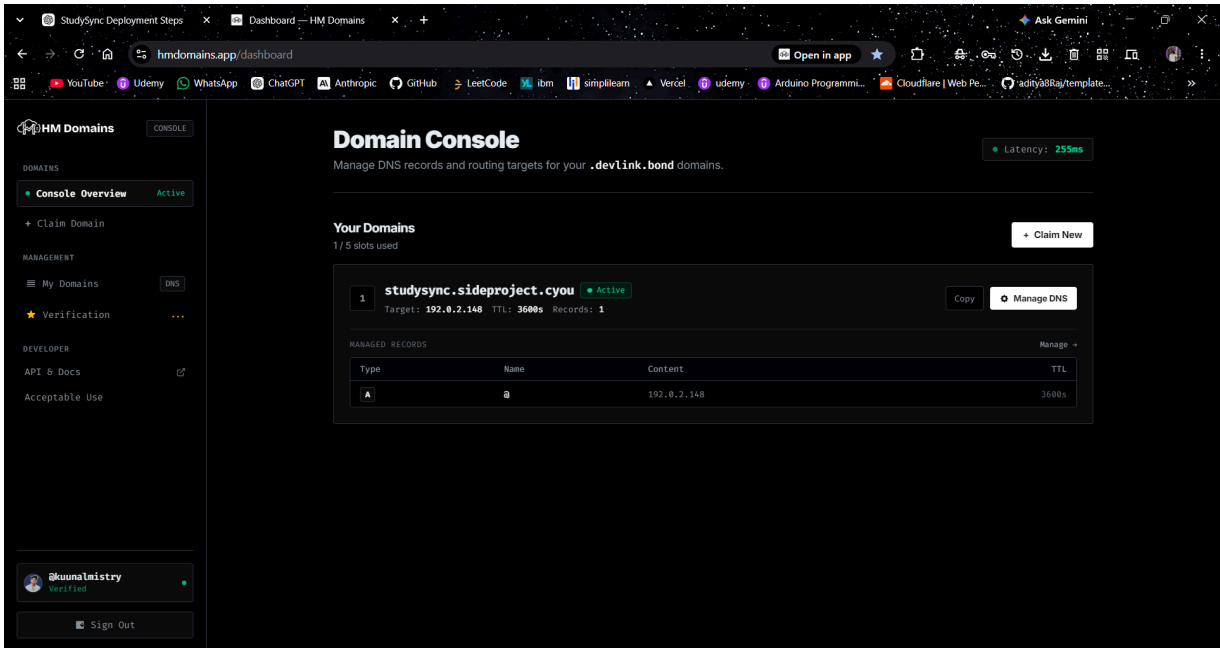
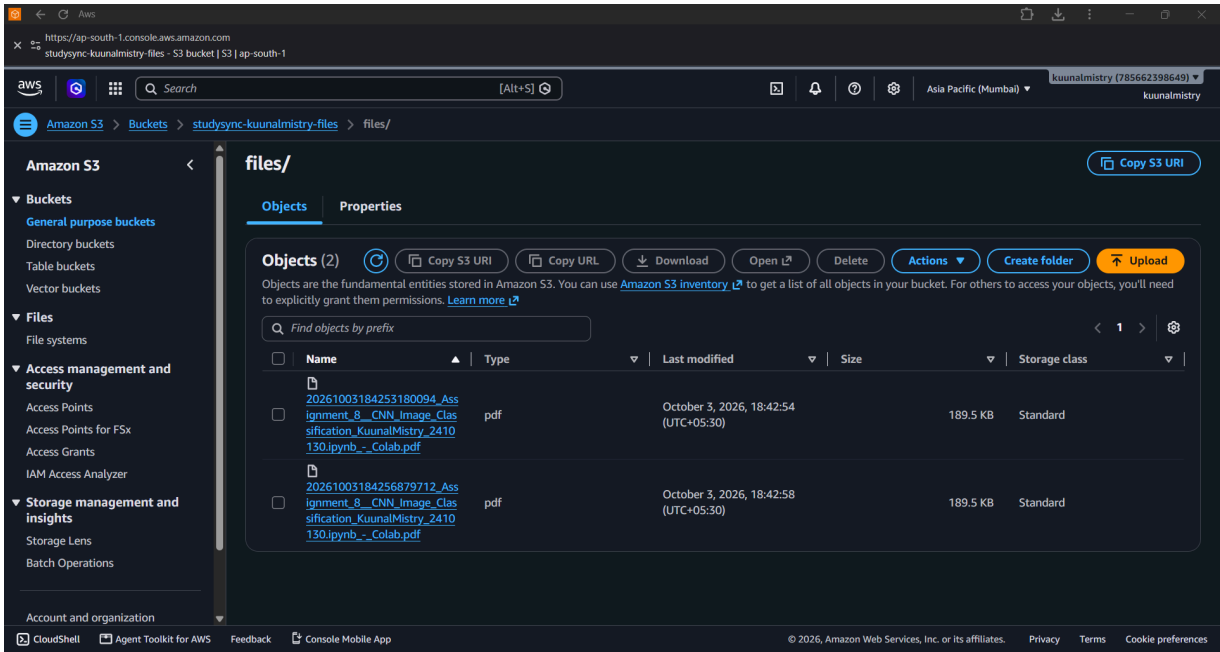
```
Flask==3.1.2
Flask-SQLAlchemy
```

Problems Output Debug Console Terminal Ports Spell Checker

```
PS C:\Users\hp\Desktop\StudySync Azure MySQL Ready\StudySync> python -c "import zipfile; z=zipfile.ZipFile('..\StudySync_Azure_Deploy.zip'); p
templates\main.html
templates\sidebar.html
templates\toasts.html
uploads\gitkeep
PS C:\Users\hp\Desktop\StudySync Azure MySQL Ready\StudySync> az webapp deployment source config-zip --resource-group StudySync-RG --name stud
ysync-kuunalmistry --src ..\StudySync_Azure_Deploy.zip
This command has been deprecated and will be removed in a future release. Use 'az webapp deploy' instead.
Getting scm site credentials for zip deployment
Starting zip deployment. This operation can take a while to complete ...
Warning up Kudu before deployment.
Deployment endpoint responded with status code 202
Polling the status of async deployment. Start Time: 2026-10-01 14:14:18.472831+00:00 UTC
Status: Building the app... Time: 0(s)
Status: Building the app... Time: 12(s)
Status: Building the app... Time: 32(s)
Status: Build successful. Time: 48(s)
Status: Starting the site... Time: 64(s)
Status: Starting the site... Time: 80(s)
Status: Starting the site... Time: 95(s)
Status: Starting the site... Time: 111(s)
Status: Starting the site... Time: 127(s)
Status: Starting the site... Time: 143(s)
Status: Starting the site... Time: 159(s)
Status: Starting the site... Time: 174(s)
Status: Starting the site... Time: 189(s)
Status: Starting the site... Time: 205(s)
Status: Starting the site... Time: 221(s)
Status: Starting the site... Time: 237(s)
Status: Starting the site... Time: 253(s)
Status: Starting the site... Time: 269(s)
Status: Starting the site... Time: 284(s)
```

Ln 7, Col 9 Spaces: 4 UTF-8 LF pip requirements Python 3.12 (64-bit) Autocomplete (0) Go Live Prettier





StudySync Deployment Steps | Workers & Pages | Kuunalmisty | Dashboard — HM Domains | Ask Gemini

dash.cloudflare.com/1ad0edfc5538e41efdae932756f039f6/workers/services/view/studysync/production

Kuunalmisty@gmail... | Workers & Pages > studysync > Production | Ask AI | Support

Quick search... Ctrl K

Overview Metrics Deployments Observability Issues Domains Access Settings

Invite Edit code Visit

studysync.kuunalmisty.workers.dev Automatic deployment on upload. by kuunalmisty 9 seconds ago

Build

- Compute
- Workers & Pages
- Observability
- Workers for Platforms
- Containers

Worker Previews New x

Every branch and pull request now gets its own preview environment.

Take a tour

Domains 1

Workers 0

Queues 0

studysync

Observability Workers Logs Workers Traces

Enabled Disabled

Bindings 0

Add a binding

No workers bound to this worker

Metrics Last 24 hours

Invocations 0 CPU Time 0 ms Errors 0

Domains and routes

workers.dev

studysync.kuunalmisty.workers.dev

StudySync Deployment Steps | Workers & Pages | Kuunalmisty | Dashboard — HM Domains | StudySync — Your Academic Workspace | Ask Gemini

dash.cloudflare.com/1ad0edfc5538e41efdae932756f039f6/workers/services/edit/studysync/production

StudySync WORKSPACE

Dashboard Assignments 2 Subjects Notes & Files Settings

Overview

Fall 2026 Academic Term

Good morning, Kuunal 🌞

CONSOLE

13:53:19 GET https://studysync.kuunalmisty.workers.dev/static/style.css

13:53:18 GET https://studysync.kuunalmisty.workers.dev/

13:53:18 GET https://studysync.kuunalmisty.workers.dev/

JS worker.js > default

```
3
4 export default {
5   async fetch(request) {
6     const incomingUrl = new URL(request.url);
7
8     const targetUrl =
9       AZURE_ORIGIN + incomingUrl.pathname + incomingUrl.search;
10
11     const proxyRequest = new Request(targetUrl, {
12       method: request.method,
13       headers: request.headers,
14       body: ["GET", "HEAD"].includes(request.method)
15         ? undefined
16         : request.body,
17       redirect: "manual",
18     });
19
20     return fetch(proxyRequest);
21   },
22 }
```

Ln 22, Col 3 Spaces: 2 UTF-8 LF JavaScript Layout: US

5. Conclusion

StudySync demonstrates the development and deployment of a cloud-based student assignment management application using a multi-cloud architecture.

The Flask application is hosted on **Azure App Service**, structured application data is stored in **Azure Database for MySQL**, uploaded files are stored in **Amazon S3**, **Cloudflare Worker** provides the public edge/reverse-proxy layer, and **Azure Application Insights** provides application monitoring.

The project successfully combines:

- Web application development
- Relational database management
- Cloud object storage
- Edge routing
- Application monitoring
- Source-code version control
- Multi-cloud service integration

The architecture demonstrates how services from multiple cloud providers can be integrated through a single web application.

The final system provides students with a centralized way to manage assignments, subjects, and academic files while demonstrating practical cloud deployment and service integration concepts covered in the **Cloud Application Development** course.

References

GitHub Repository: <https://github.com/kuunalmistry/StudySync>

Live StudySync Application: <https://studysync.kuunalmistry.workers.dev>

